

Januar 2024.

5.

a) *minimalni skup operatora relacije algebre*: projekcija, restrikcija, proizvod, unija, razlika

još neki operatori: Dekartov proizvod, presek, (prirodno) spajanje, deljenje

b) *Prednosti rada sa bazom podataka*:

- Podaci se mogu deliti između postojećih aplikacija, a takođe se mogu praviti nove aplikacije koje će koristiti postojeću bazu podataka bez njene izmene

- Smanjuje se redundantnost podataka, odnosno podaci se ponavljaju samo u onoj meri koja je neophodna za efikasan rad

- Uklanjanjem redundantnosti eliminiše se i nekonzistentnost, odnosno ako imamo neki podatak na više mesta, ako ga izmenimo na jednom neće se izmeniti na ostalim i dolazi do greške

- Postoji podrška za rad sa transakcijama. Transakcija je logička jedinica posla nad bazom podataka. Karakteriše je atomičnost – sve operacije u okviru jedne transakcije su izvršene kako treba ili se nijedna neće izvršiti

- Održavanje integriteta – obezbeđivanje da su podaci u bazi tačni. Postiže se definisanjem pravila koja moraju da budu zadovoljena u svakom trenutku (npr. na ispitu može da se dobije samo ocena između 5 i 10)

- Primena zaštite podataka – ne mogu svi korisnici da vide sve podatke

- Balansiranje između konfliktnih zahteva – podaci u bazi su strukturirani tako da zadovolje potrebe svih korisnika, ne pojedinaca

- primena standarda – internacionalni, industrijski, u okviru organizacije kojoj baza pripada

v) *detaljno opisati efekat referencijalnih akcija koje mogu da se jave pri operaciji ažuriranja/unosa/brisanja podataka u sistemu DB2*

Referencijalne akcije su **pravila koja se primenjuju kada se promeni vrednost u tabeli koja je u vezi sa drugom tabelom** putem **stranog ključa (FOREIGN KEY)**.

Referencijalne akcije se javljaju kod sledećih operacija:

1. **INSERT** – Kada se pokušava dodati novi red koji referencira drugi red (npr. student koji se vezuje za nepostojeći fakultet).
2. **UPDATE** – Kada se menja vrednost **primarnog ključa** u roditeljskoj tabeli, ili stranog ključa u povezanoj tabeli.
3. **DELETE** – Kada se briše red iz **roditeljske** (referencirane) tabele.

Vrste referencijalnih akcija (u DB2 i većini SGBD-a):

1. NO ACTION

- **Podrazumevano ponašanje.**
- **Zabranjuje** brisanje ili ažuriranje ako postoje povezani redovi u drugoj tabeli.
- **Efekat:** Transakcija se prekida i baca grešku (SQLSTATE greška).

Primer:

Ako pokušamo da obrišemo fakultet koji ima upisane studente → baza neće dozvoliti brisanje.

2. CASCADE

- Promena u roditeljskoj tabeli **automatski se prenosi** na povezane redove u zavisnoj tabeli.

Efekat:

- DELETE CASCADE → Brisanjem reda u roditeljskoj tabeli, **automatski se brišu** svi povezani redovi u zavisnoj tabeli.
- UPDATE CASCADE → Ažuriranjem ključa u roditeljskoj tabeli, **automatski se ažurira i strani ključ** u zavisnoj tabeli.

Primer:

Ako obrišemo fakultet → svi studenti s tog fakulteta se takođe brišu.

3. SET NULL

- Kada se obriše ili ažurira red u roditeljskoj tabeli, **odgovarajući strani ključ u detetu se postavlja na NULL.**

Uslov: Strani ključ mora dozvoliti NULL vrednosti.

Primer:

Ako obrišemo fakultet, IDFakulteta u tabeli Student postaje NULL.

4. RESTRICT

Restrict akcija ima isti efekat kao i akcija no action. Navedene dve akcije se razikuju ukoliko na roditeljsku tabelu referiše više dete tabela, jer se pri pokušaju brisanja prvo proveravaju zavisne tabele sa definisanom restrict akcijom, a poslednje zavisne tabele sa definisnom no action akcijom.

g) *Navedite bar 8 agregatnih funkcija koje poznajete. Kako se ponašaju pri radu sa funkcijama koje imaju null vrednost?*

Agregatne funkcije se koriste kada je potrebno izvršiti neke operacije nad svim entitetima koji ulaze u rezultat upita.

1. **COUNT(*)** – Broji sve redove, uključujući i one koji sadrže NULL.
2. **COUNT(column)** – Broji samo redove u kojima **column nije NULL.**
3. **SUM(column)** – Vraća zbir vrednosti kolone (ignorisan NULL).
4. **AVG(column)** – Računa prosečnu vrednost kolone (ne uključuje NULL).

5. **MIN(column)** – Najmanja vrednost kolone (preskače NULL).
6. **MAX(column)** – Najveća vrednost kolone (preskače NULL).
7. **VARIANCE(column)** ili **VAR_POP** – Statistička varijansa kolone.
8. **STDDEV(column)** – Standardna devijacija (mera raspršenja podataka).
9. **LISTAGG(column, ',')** – Spaja više vrednosti u jedan string, razdvojene zadatim separatorom.
10. **MEDIAN(column)** – Srednja vrednost skupa (nije dostupna u svim sistemima)
11. **correlation** – koristi se za racunanje korelacije vrednosti zadatih kolona numeričkog tipa; koristi se u obliku **correlation(ime-kolone1, ime-kolone2)**

Sve **osim COUNT(*)** ignorišu **NULL** vrednosti.

6. a)

```
CREATE FUNCTION POVRSINA (
  UGAO DOUBLE,
  POLUPRECNIK DOUBLE
)
RETURNS VARCHAR(100)
BEGIN
  DECLARE POVRSINA_ISECKA DOUBLE;
  DECLARE REZULTAT VARCHAR(100);
  -- Provera ispravnosti argumenata
  IF UGAO <= 0 THEN
    SET REZULTAT = 'Greska: Ugao mora biti veci od 0';
  ELSEIF POLUPRECNIK <= 0 THEN
    SET REZULTAT = 'Greska: Poluprecnik mora biti veci od 0';
  ELSE
    SET POVRSINA_ISECKA = (POLUPRECNIK * POLUPRECNIK * UGAO) / 2;
    SET REZULTAT = 'Povrsina iznosi: ' || CHAR(POVRSINA_ISECKA);
  END IF;
  RETURN REZULTAT;
END
```

b) izbačen iz gradiva

7. a)

- 1) $CG \rightarrow B, CG \rightarrow D$
- 2) $AC \rightarrow D$
- 3) $BFG \rightarrow D$
- 4) $ACG \rightarrow E, ACG \rightarrow F$
- 5) $BD \rightarrow F, BD \rightarrow G$
- 6) $G \rightarrow AD, G \rightarrow D$
- 7) $EG \rightarrow B, EG \rightarrow F$
- 8) $DG \rightarrow C$

b) Kandidati za ključ

R = {A, B, C, D, E, F}

Tražimo skup čije **zatvorenje daje svih 6 atributa**.

Testiramo zatvorenja:

Zatvorenje G^+

$G \rightarrow A, D \rightarrow$ dodamo A, D

$G, D \rightarrow C$

$G, A, D, C \rightarrow$

$\rightarrow ACG \rightarrow E, F$

$\rightarrow CG \rightarrow B$

$G^+ = \{A, B, C, D, E, F\} = R$

Znači G je ključ? Ali G je samo jedan atribut $\rightarrow$ proverimo minimalnost

$\rightarrow$ Može li manji skup od G generisati sve? Nema manjih $\rightarrow$ G je **kandidat** (ali moraš proveriti da li je **samo G**)

Da li je G minimalan?

- Proveri da $G^+ = R \rightarrow$ već jesmo
- Nema podskupa G $\rightarrow$ jer G je sam po sebi atribut $\rightarrow$ **da, G je minimalan ključ**

Kandidat ključ: {G}

v) Iz F3, zadržaćemo samo one zavisnosti gde se **svi atributi (leva i desna strana)** nalaze unutar R1.

- $AC \rightarrow D$
- $ACG \rightarrow E$

8. a) Kod konkurentnih transakcija, dolazi do problema pojave fantoma. A čita neke redove iz tabele, B upisuje neke redove u istu tabelu, zatim A opet čita redove iz tabele po istom uslovu, pronalazi nove redove koji u prvom čitanju nisu bili tu, ali ih je B u međuvremenu upisala, takozvane **fantomske redove**.

- (pisanje, čitanje) - A menja t, a zatim B želi da čita t. U ovoj situaciji može doći do problema zavisnosti od nepotvrđenog čitanja. Ukoliko je dozvoljeno da B čita nepotvrđenu vrednost, to se još zove i **prljavo čitanje**.

- (pisanje, pisanje) - A menja t, a zatim B želi da menja t. U ovoj situaciji može doći do problema izgubljenog ažuriranja. Ukoliko je dozvoljeno da B menja vrednost koja može biti poništena, to se još zove i **prljavo pisanje**.

b)

- formiraju se dve liste transakcije UNDO i REDO

-u UNDO se dodaju sve transakcije koje su bile aktivne u poslednjem checkpoint-u

- Od poslednjeg checkpoint-a ide se napred log datotekom: za BEGIN TRANSACTION log dodaje se transakcija u listu UNDO transakcija, a za COMMIT log premešta se transakcija iz UNDO u REDO

- Kada se stigne do kraja log datoteke kreće se nazad ka poslednjem checkpoint-u i za svaku transakciju iz liste UNDO se izvršava undo operacija

-Kreće se od poslednjeg checkpoint-a ka kraju log datoteke i za svaku transakciju iz liste REDO se izvršava redo operadica

v) *Algoritam za unošenje podataka u B+ drvo:*

- Odrediti korektan list L.
- Upisati novu vrednost u list L.
- Ako postoji prostor unos je završen.
- Ako ne postoji prostor
 - Dodaje se novi list L1 i podaci iz lista L se ravnomerno raspoređuju na L i L1 tako da je svaki od njih pun bar do polovine kapaciteta.
 - Unosi se bočni pokazivač sa L na L1 i sa L1 na prethodni naredni od L.
 - Srednji ključ se prepisuje u čvor na nivou iznad.
 - Unosi se pokazivač na list L1 sa roditelj čvora lista L.
- Podela čvora na jednom nivou utiče na čvor na višem nivou ako u njega treba da se unese pokazivač na ključ. Po potrebi, rekurzivno ponoviti postupak – ako postoji prostor uneti vrednost/pokazivač, a ako ne postoji podeliti čvor.
- Ako se deli indeksni čvor, podaci se ravnomerno raspoređuju, ali se srednji ključ pomera naviše.
- Izuzetak predstavlja koreni čvor – ako treba da se u njega unese podatak a ne postoji slobodan prostor, tada se koreni čvor deli na dva i formira novi koreni čvor na višem nivou sa dva dete čvora.
- Bez obzira na broj n korenom čvoru je dozvoljeno da ima samo jedan ključ i dva dete čvora.

FEBRUAR 2024.

5. a) Nezavisnost podataka u bazama podataka je ključni koncept koji omogućava da se promene u načinu skladištenja ili organizacije podataka ne odraze direktno na aplikacije koje koriste te podatke. Omogućava **fleksibilnost i održavanje** baza podataka. Ona je ključni razlog zašto se koriste **sistemi za upravljanje bazama podataka (DBMS)** – jer obezbeđuju da su logički, konceptualni i fizički nivoi odvojeni.

b) Za upitni jezik je potrebno da je **relaciono kompletan**, odnosno da je moćan isto kao i algebra, tj. da bilo koja relacija predstavljiva u relacionoj algebri može da se predstavi i u upitnom jeziku. Upitni jezik **SQL je relaciono kompletan** jer postoje SQL izrazi za svaki od 5 primitivnih operatora relacione algebra

v) Integritet baze podataka se odnosi na to da postoje unapred definisani uslovi za attribute u bazi podataka (npr. ocena je od 5 do 10 i ne može se uneti 4), a **sigurnost** se odnosi na pravo pristupa – može da se ograniči koji korisnik može kom delu baze da pristupi.

Vrste ključeva: **primarni** (jedan ili više atributa koji jedinstveno identifikuju svaki red u tabeli, jedan od kandidata za ključ), **alternativni** (svi kandidati za ključ sem primarnog), **strani** (skup atributa relacije R2 čije vrednosti odgovaraju vrednostima nekog kandidata za ključ relacije R1), **superključ** (nadskup kandidata za ključ, poseduje jedinstvenost, ali ne i minimalnost)

6.

a) izbačeno

b) Ažuriranje pogleda ne sme da naruši ograničenja integriteta nad pogledima.

- from klauzula sadrži referencu na tačno jednu tabelu koja je ili osnovna tabela ili pogled koji može da se ažurira
- Izraz kojim se definiše pogled je select izraz koji ne sadrži JOIN, UNION, INTERSECT ili EXCEPT
- select klauzula ne sadrži ključnu reč DISTINCT
- svaka select stavka sadrži ime koje predstavlja referencu na kolonu osnovne tabele
- where klauzula select izraza ne sadrži podupit u kome se from klauzula referiše na istu tabelu kao i from klauzula u select izrazu na najvišem nivou
- select izraz ne sadrži group by niti having klauzulu

v) CREATE VIEW DA.STUDENTI_PREKO_DEVET AS

SELECT

d.Ime,

d.Prezime,

d.BrojIndeksa

FROM

DA.Dosije d

JOIN

DA.Ispit i ON d.BrojIndeksa = i.BrojIndeksa

WHERE

i.Status = 'o'

GROUP BY

d.Ime, d.Prezime, d.BrojIndeksa

HAVING

$SUM(i.Ocena) * 1.0 / COUNT(i.Ocena) > 9.00;$

Nije moguće ažuriranje pogleda zbog JOIN, GROUP BY, HAVING, SUM, COUNT.

7. a) $A \rightarrow B, A \rightarrow C, A \rightarrow D, DE \rightarrow G, DE \rightarrow H, E \rightarrow F, D \rightarrow C, AG \rightarrow F, EC \rightarrow F, EC \rightarrow G, D \rightarrow B, D \rightarrow E$

b) A je jedini kandidat za ključ pošto kada se krene od bilo kog drugog ne može da se stigne do A

v) **PODSEĆANJE:**

Funkcionalna zavisnost $X \rightarrow Y$ relacije R, gde su X i Y podskupovi skupa atributa relacije R, je:

- superključna ili potpuna ukoliko je X ceo ključ relacije R
- trivijalna ukoliko je Y podskup od X
- parcijalna ukoliko je X pravi podskup ključa relacije R
- tranzitivna ukoliko postoji podskup Z skupa atributa relacije R različit od X i Y takav da važi $X \rightarrow Z$ i $Z \rightarrow Y$.

FZ	Tip FZ
$A \rightarrow B$	Superključna
$A \rightarrow C$	Superključna
$A \rightarrow D$	Superključna
$DE \rightarrow G$	Tranzitivna
$DE \rightarrow H$	Tranzitivna
$E \rightarrow F$	Tranzitivna
$D \rightarrow C$	Tranzitivna
$AG \rightarrow F$	Superključna
$EC \rightarrow G$	Tranzitivna ili parcijalna
$D \rightarrow B$	Tranzitivna
$D \rightarrow E$	Tranzitivna

Da li je R u BCNF?

Definicija BCNF:

Relacija je u BCNF ako za svaku FZ $X \rightarrow Y$ važi:

- X je superključ (tj. sadrži ključ)

Kako nije svaka FZ superključna, R nije u BCNF.

Dekompozicija u BCNF

Korak 1: Izaberemo FZ koja krši BCNF, npr. $E \rightarrow F$

Razdvajamo R na dve relacije:

- $R_1 = (E, F)$
- $R_2 = R \setminus \{F\} = \{A, B, C, D, E, G, H\}$

Korak 2: U R2 proveravamo ostatak zavisnosti, i dalje proveravamo BCNF

Korak 3: Dalja dekompozicija za $D \rightarrow B, E$

Razdvajamo po zavisnosti $D \rightarrow B, E$:

- $R_3 = (D, B, E, C)$ (dodato C jer $D \rightarrow C$)
- $R_4 =$ ostalo

Dalje dekompozicije su moguće za ostale zavisnosti koje krše BCNF.

Očuvanje FZ:

- Neki od skupova FZ **nisu očuvani** jer dekompozicija može da ukloni neke zavisnosti ako nisu pokrivene u novim relacijama.

8. a) Konkurentnost kod transakcija podrazumeva da više transakcija ima pristup istim podacima istovremeno. Problemi koji mogu nastati su:

- **Problem nekonzistentne analize** (jedna transakcija prolazi kroz podatke dok ih druga transakcija menja i na kraju ne dobije dobru vrednost, npr. sabira vrednosti na računima dok neko dodaje ili skida pare)
- **Problem zavisnosti od nepotvrđenog čitanja** (Jedna transakcija pročita podatke koje je neka druga transakcija **promenila, ali još nije potvrdila (commit)**. Ako druga transakcija kasnije **vraća te promene nazad (rollback)**, prva transakcija se oslanjala na **nevažeće** podatke)
- **Problem izgubljenog ažuriranja** (Dve transakcije **menjaju isti podatak**, ali se druga transakcija **prepiše preko prve**, pa se **izgubi** rezultat prvog ažuriranja)
- **Problem neuzastopnih čitanja** (A čita t, a zatim B želi da izmeni t. Ukoliko B izmeni t, a zatim A pročita ponovo, dobiće drugaciju vrednost što je problem neuzastopnih čitanja)
- **Problem pojavljivanja fantoma** (Jedna transakcija pokrene **upit koji vraća određeni broj redova**, ali ako ga ponovi kasnije, **dobije drugačiji broj redova**, jer je druga transakcija **ubacila (INSERT)** nove redove koji zadovoljavaju isti uslov)

b) Transakcija je logička jedinica posla. Može da sadrži niz operacija, ali koje bi trebalo da se izvrše atomično, odnosno ili sve da se izvrše ili nijedna da se ne izvrši. Takođe, ne smeju se naredbe izvršiti nad nekim slogovima, a nad nekim ne nad kojim je planirano izvršenje, već se izvršavaju nad svim planiranim ili ni nad jednim. Deo sistema RDBMS koji se zove upravljač transakcijama obezbeđuje atomično izvršavanje operacija u transakciji. On obezbeđuje da se transakcije ne izgube, da ne budu delimično izvršene ili da ne budu izvršene više puta.

Transakcije poseduju ACID osobine:

- Atomicnost - sve naredbe se izvršavaju ili ni jedna
- Konzistentnost - baza nakon izvršenja transakcije treba da bude u konzistentnom stanju
- Izolovanost - transakcije nemaju uvid u međurezultate drugih transakcija u toku izvršavanja
- Trajnost - po uspešnom završetku transakcije promene ostaju u bazi i mogu se izmeniti samo drugom transakcijom

Transakcije omogućavaju:

- **Konzistentan i tačan rad** više korisnika/programa u isto vreme
- **Sprečavanje problema konkurentnog pristupa**, kao što su:
 - Nepotvrđeno čitanje
 - Izgubljeno ažuriranje
 - Fantomi
 - Neponovljeno čitanje

v) **brisanje B+ drveta**

- Odrediti korektan list L u kome se nalazi ključ koga treba obrisati.
- Ako posle uklanjanja ključa list ima bar polovinu elemenata, brisanje je završeno.
- U suprotnom, ako neki od susednih listova ima više od minimalnog broja ključeva i pokazivača, tada se od njega pozajmljuje par ključ/pokazivač, tako da se redosled ključeva ne menja. Pri tome se vodi računa da roditelj ključevi treba da pokazuju na novo stanje.

- Ako oba susedna lista imaju minimum parova, tada se spajaju dva susedna lista i briše jedan od njih. Broj ključeva/pokazivača u novom listu je manji od maksimuma (spojeni su listovi sa minimumom i manje od minimuma broja parova).
- Ažuriraju se ključevi roditelj čvora spojenih listova i izbriše par ključ/pokazivač u roditelj čvoru koji je pokazivao na izbrisani list.
- Ako roditelj čvor ima manje od minimalnog broja parova, postupak se rekurzivno ponavlja, u suprotnom brisanje je završeno